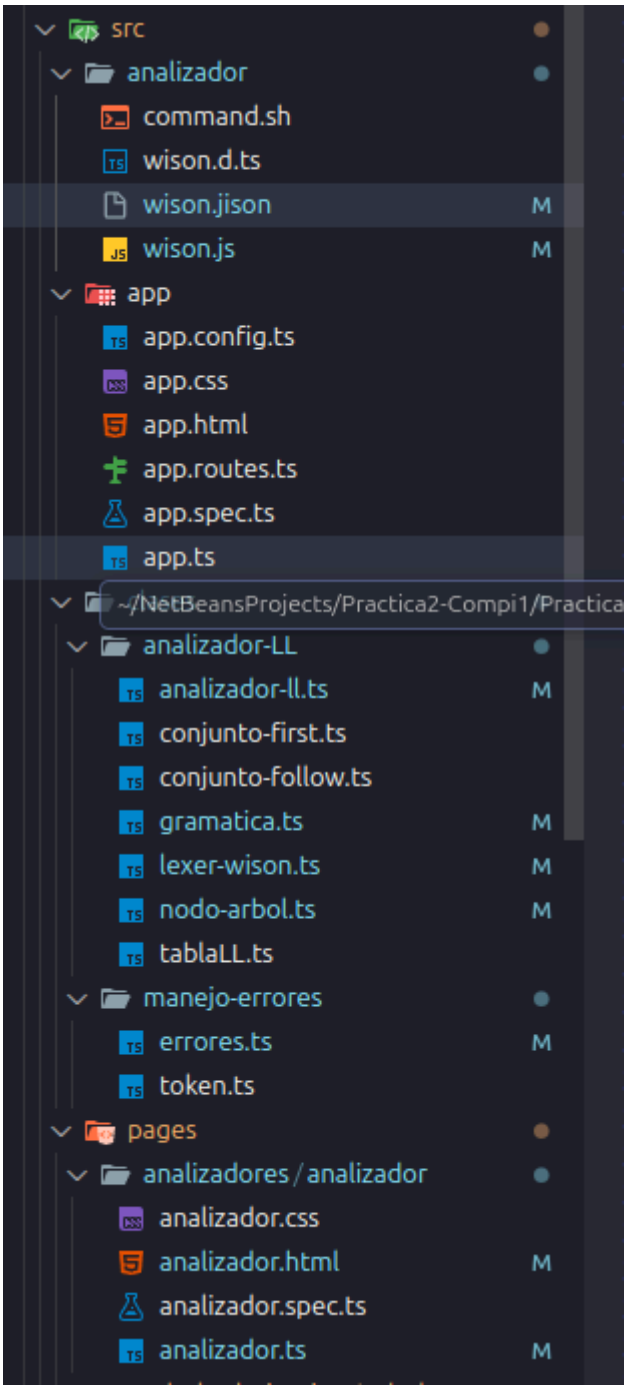
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: ORGANIZACIÓN DE LENGUAJES Y COMPILADORES
CATEDRÁTICO: ING. MOISES GRANADOS

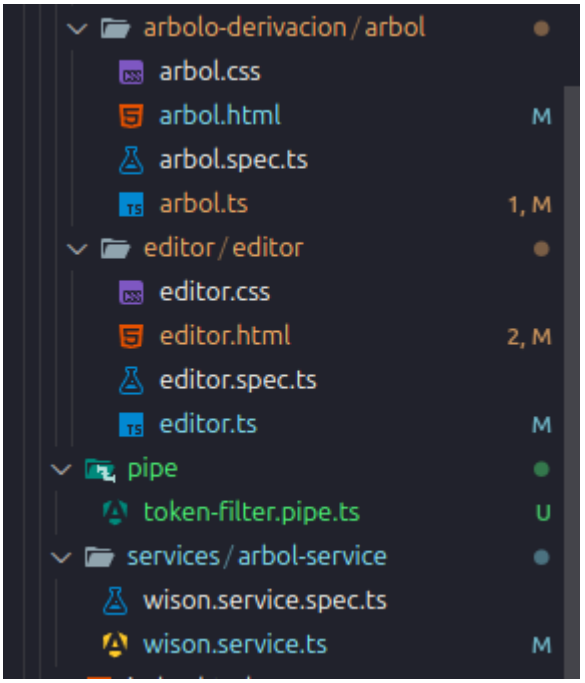
PROYECTO 1: GENERADOR DE ANALIZADORES LL
MANUAL DE TECNICO

ESTUDIANTE	**CARNÉ:**
Cristian Alejandro Roldán López	202147280

QUETZALTENANGO, 09 DE ABRIL 2,026

Organización del proyecto





analizador: En este apartado se genera el archivo .js que contiene el analizador generado por jison, en este caso se tiene un script para compilar el archivo jison que genera el archivo wison.js.

app: Aca se colocan las rutas para navegar entre cada página creada en el proyecto en el apartado de app.routes.ts una de las funciones principales que tiene.

analizador-LL: Se tiene la lógica para crear la tabla de símbolos, los nodos del árbol, calcular el siguiente y primero cuando se recorre la gramática para generar el analizador,

lexer-wison que hace la tokenización de las expresiones regulares del lexer en el formato wison para poder utilizarlo cuando se valida una cadena.

manejo-errores: errores.ts archivo donde se validan y guardan las producciones generadas en wison para saber si es un expresión, terminal, no terminal y producciones, también se validan errores léxicos, sintácticos y semánticos. token.ts clase para la estructura que tienen los tokens encontrados.

pages: Aca se encuentra la interfaz que visualiza el usuario, el editor para generar las gramáticas, el analizador para las cadenas y utilizar los analizadores creados y la visualización del árbol de derivación.

pipe: Clase creada para saber que tipo de token es el que viene, se utiliza para la tabla de símbolos y los errores cuando se analiza una cadena utilizando algún analizador.

arbol-service: Service para construir el AST, analizar las cadenas o textos, eliminar los analizadores creados, guardar y cargar los analizadores esto se hace en el localStorage para mantener la persistencia.

Gramática Jison

Parte léxica.

```
%lex
%options case-insensitive
%%

\s+                                          /* ignorar espacios */
"#".*                                        /* comentario línea */
[/][*][*]([^*]|[*]+[^*/])*[*]+[/]           /* comentario multilinea */

"[0-9]"                 return 'RANGO_NUMERO';
"[a-z]"                 return 'RANGO_LETRA_MINUSCULA';
"[A-Z]"                 return 'RANGO_LETRA_MAYUSCULA';
"[aA-zZ]"               return 'RANGO_ABECEDARIO';

"Wison"                 return 'WISON';
"Lex"                   return 'LEX';
"Terminal"              return 'TERMINAL';
"No_Terminal"           return 'NO_TERMINAL';
"Syntax"                return 'SYNTAX_PARSER';
"Initial_Sim"           return 'SIMBOLO_INICIAL';

"<="                    return 'FLECHA';
```

```
"<-"                    return 'ASIGNACION';

"{{:"                   return 'SYNTAX_BLOQUE_ABRE';
":}}"                   return 'SYNTAX_BLOQUE_CIERRA';
"{:"                    return 'BLOQUE_ABRE';
":}"                    return 'BLOQUE_CIERRA';

"("                     return 'PARENTESIS_ABRE';
")"                     return 'PARENTESIS_CIERRA';
"["                     return 'CORCHETE_ABRE';
"]"                     return 'CORCHETE_CIERRA';
";"                     return 'PUNTO_COMA';
"|"                     return 'OR';
"+"                     return 'MAS';
"*"                     return 'KLEENE';
"¿"                     return 'APERTURA';

"?Wison"                return 'CIERRE_WISON';
"?"                     return 'OPCIONAL';

['](([^'])*[']          return 'CADENA';

"$_"[a-zA-Z0-9_]+     return 'ID_TERMINAL';
"%_"[a-zA-Z0-9_]+     return 'ID_NO_TERMINAL';

<<EOF>>                 return 'EOF';

. {
   yy.manejador.errorLexico(
       yytext,
       yylloc.first_line,
       yylloc.first_column + 1
   );
}

/lex
```

Parte sintáctica.

```
%start inicio
%%

inicio
    : WISON APERTURA bloque_lex bloque_syntax CIERRE_WISON EOF
    {
        $$ = {
            tipo: 'Wison',
            lex: $3,
            syntax: $4,
            linea: @1.first_line,
            columna: @1.first_column
        };
        return {
            ast: $$,
            tokens: yy.manejador.getToken(),
            errores: yy.manejador.getError()
        };
    }
    | error EOF
    {
        yy.manejador.errorEstructuraWison(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );

        return {
            ast: null,
            tokens: yy.manejador.getToken(),
            errores: yy.manejador.getError()
        };
    }
    ;

/* ———————————— BLOQUE LEX ———————————— */

bloque_lex
    : LEX BLOQUE_ABRE lista_terminales BLOQUE_CIERRA
```

```
    {
        $$ = {
            tipo: 'BloqueLexico',
            terminales: $3,
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    | LEX BLOQUE_ABRE BLOQUE_CIERRA
    {
        $$ = {
            tipo: 'BloqueLexico',
            terminales: [],
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    | error BLOQUE_CIERRA
    {
        yy.manejador.errorBloqueLex(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );
        $$ = { tipo: 'BloqueLexico', terminales: [], linea:
@1.first_line, columna: @1.first_column };
    }
    ;

lista_terminales
    : lista_terminales declaracion_terminal
    {
        $$ = $1;
        $$.push($2);
    }
    | declaracion_terminal
    {
        $$ = [$1];
    }
    ;

declaracion_terminal
    : TERMINAL ID_TERMINAL ASIGNACION expresion_regular PUNTO_COMA
```

```
    {
        $$ = {
            tipo: 'DeclaracionTerminal',
            nombre: $2,
            expresion: $4,
            linea: @1.first_line,
            columna: @1.first_column
        };
        yy.manejador.agregarToken(
            $2,
            @2.first_line,
            @2.first_column + 1,
            "Expresión regular: " + yy.manejador.regexToString($4),
            "TERMINAL"
        );

    }
    | error PUNTO_COMA
    {
        yy.manejador.errorDeclaracionTerminal(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );
        $$ = null;
    }
    ;

/* ———————————— EXPRESIONES REGULARES ———————————— */

expresion_regular
    : expresion_regular OR expresion_concatenada
    {
        $$ = { tipo: 'Union', izq: $1, der: $3, linea: @1.first_line,
columna: @1.first_column };
    }
    | expresion_concatenada
    {
        $$ = $1;
    }
    ;

expresion_concatenada
```

```
    : expresion_concatenada expresion_unaria
    {
        $$ = { tipo: 'Concatenacion', izq: $1, der: $2, linea:
@1.first_line, columna: @1.first_column };
    }
    | expresion_unaria
    {
        $$ = $1;
    }
    ;

expresion_unaria
    : expresion_primaria KLEENE
    {
        $$ = { tipo: 'Kleene', expr: $1, linea: @1.first_line, columna:
@1.first_column };
    }
    | expresion_primaria MAS
    {
        $$ = { tipo: 'CerraduraPositiva', expr: $1, linea:
@1.first_line, columna: @1.first_column };
    }
    | expresion_primaria OPCIONAL
    {
        $$ = { tipo: 'Opcional', expr: $1, linea: @1.first_line,
columna: @1.first_column };
    }
    | expresion_primaria
    {
        $$ = $1;
    }
    ;

expresion_primaria
    : CADENA
    {
        $$ = { tipo: 'Caracter', valor: $1.slice(1,-1), linea:
@1.first_line, columna: @1.first_column };
    }
    | RANGO_NUMERO
    {
        $$ = { tipo: 'Rango', valor: '[0-9]', linea: @1.first_line,
columna: @1.first_column };
```

```
    }
    | RANGO_LETRA_MINUSCULA
    {
        $$ = { tipo: 'Rango', valor: '[a-z]', linea: @1.first_line,
columna: @1.first_column };
    }
    | RANGO_LETRA_MAYUSCULA
    {
        $$ = { tipo: 'Rango', valor: '[A-Z]', linea: @1.first_line,
columna: @1.first_column };
    }
    | RANGO_ABECEDARIO
    {
        $$ = { tipo: 'Rango', valor: '[aA-zZ]', linea: @1.first_line,
columna: @1.first_column };
    }
    | ID_TERMINAL
    {
        $$ = { tipo: 'ReferenciaTerminal', nombre: $1, linea:
@1.first_line, columna: @1.first_column };
    }
    | PARENTESIS_ABRE expresion_regular PARENTESIS_CIERRA
    {
        $$ = { tipo: 'Grupo', expr: $2, linea: @1.first_line, columna:
@1.first_column };
    }
    | error
    {
       yy.manejador.errorExpresionRegular(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );

        $$ = { tipo: 'Error', linea: @1.first_line, columna:
@1.first_column };
    }
    ;

/* ———————————— BLOQUE SYNTAX ———————————— */

bloque_syntax
```

```
    : SYNTAX_PARSER SYNTAX_BLOQUE_ABRE lista_no_terminales
simbolo_inicial lista_producciones SYNTAX_BLOQUE_CIERRA
    {
        $$ = {
            tipo: 'BloqueSintactico',
            noTerminales: $3,
            simboloInicial: $4,
            producciones: $5,
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    | SYNTAX_PARSER SYNTAX_BLOQUE_ABRE lista_no_terminales
lista_producciones SYNTAX_BLOQUE_CIERRA
    {
        yy.manejador.errorSintactico(
            '',
            @1.first_line,
            @1.first_column + 1,
            'Falta declarar el simbolo inicial: Initial_Sim %_Nombre ;'
        );
        $$ = {
            tipo: 'BloqueSintactico',
            noTerminales: $3,
            simboloInicial: null,
            producciones: $4,
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    | SYNTAX_PARSER SYNTAX_BLOQUE_ABRE SYNTAX_BLOQUE_CIERRA
    {
        $$ = {
            tipo: 'BloqueSintactico',
            noTerminales: [],
            simboloInicial: null,
            producciones: [],
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    | error SYNTAX_BLOQUE_CIERRA
    {
```

```
        yy.manejador.errorBloqueSyntax(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );

        $$ = {
            tipo: 'BloqueSintactico',
            noTerminales: [],
            simboloInicial: null,
            producciones: [],
            linea: @1.first_line,
            columna: @1.first_column
        };
    }
    ;

lista_no_terminales
    : lista_no_terminales declaracion_no_terminal
    {
        $$ = $1;
        $$.push($2);
    }
    | declaracion_no_terminal
    {
        $$ = [$1];
    }
    ;

declaracion_no_terminal
    : NO_TERMINAL ID_NO_TERMINAL PUNTO_COMA
    {
        $$ = {
            tipo: 'DeclaracionNoTerminal',
            nombre: $2,
            linea: @1.first_line,
            columna: @1.first_column
        };
        yy.manejador.agregarToken(
            $2,
            @2.first_line,
            @2.first_column + 1,
            "No Terminal declarado",
```

```
            "NO_TERMINAL"
        );

    }
    ;

simbolo_inicial
    : SIMBOLO_INICIAL ID_NO_TERMINAL PUNTO_COMA
    {
        $$ = {
            tipo: 'SimboloInicial',
            nombre: $2,
            linea: @1.first_line,
            columna: @1.first_column
        };
        yy.manejador.agregarToken(
            $2,
            @2.first_line,
            @2.first_column + 1,
            "Simbolo de inicio",
            "SIMBOLO_INICIAL"
        );
    }
    ;



lista_producciones
    : lista_producciones produccion
    {
        $$ = $1;
        $$.push($2);
    }
    | produccion
    {
        $$ = [$1];
    }
    ;

produccion
    : ID_NO_TERMINAL FLECHA cuerpo_produccion PUNTO_COMA
    {
```

```
        let textoProduccion =
            $1 + " → " +
            yy.manejador.produccionToString($3);

        $$ = {
            tipo: 'Produccion',
            cabeza: $1,
            cuerpo: $3
        };

        yy.manejador.agregarToken(
            textoProduccion,
            @1.first_line,
            @1.first_column + 1,
            "Producción gramatical",
            "PRODUCCION"
        );

    }
    | error PUNTO_COMA
    {
        yy.manejador.errorProduccion(
            yytext,
            @1.first_line,
            @1.first_column + 1
        );

        $$ = null;
    }
    ;

cuerpo_produccion
    : cuerpo_produccion OR lista_simbolos
    {
        $$ = {
            tipo:'Alternativa',
            opciones:[
                ...($1.tipo=='Alternativa' ? $1.opciones : [$1]),
                $3
            ]
        };
    }
    | lista_simbolos
```

```
    {
        $$ = $1;
    }
    | /* ε */
    {
        $$ = {
            tipo:'Secuencia',
            simbolos:[],
            epsilon:true
        };
    }
    ;

lista_simbolos
    : lista_simbolos simbolo
    {
        $$ = { tipo: 'Secuencia', simbolos: [...($1.tipo === 'Secuencia'
? $1.simbolos : [$1]), $2] };
    }
    | simbolo
    {
        $$ = { tipo: 'Secuencia', simbolos: [$1] };
    }
    ;

simbolo
    : ID_TERMINAL
    {
        $$ = { tipo: 'SimboloTerminal', nombre: $1, linea:
@1.first_line, columna: @1.first_column };
    }
    | ID_NO_TERMINAL
    {
        $$ = { tipo: 'SimboloNoTerminal', nombre: $1, linea:
@1.first_line, columna: @1.first_column };
    }
    ;
```

Clase manejo de errores y forma de guardar los tokens.

```
import { Token } from './token';

/* Clase para manejar los tokens y errores del análisis léxico y
sintáctico */

export class ManejoErrores {

 private tokens: Token[] = [];
 private errores: Token[] = [];

 reset() {
   this.tokens = [];
   this.errores = [];
 }

 agregarToken(lexema: string, linea: number, columna: number,
descripcion: string, tipo: string) {
   this.tokens.push(new Token(lexema, linea, columna, descripcion,
tipo));
 }

 /* ERRORES LEXICOS */

 errorLexico(lexema: string, linea: number, columna: number) {
   this.errores.push(new Token(
     lexema, linea, columna,
     `Caracter no reconocido: '${lexema}'`,
     'LEXICO'
   ));
 }

 /* ERRORES SINTACTICOS */

 errorSintactico(lexema: string, linea: number, columna: number,
descripcion: string) {
   this.errores.push(new Token(
     lexema,
     linea,
     columna,
     descripcion,
     'SINTACTICO'
```

```
    ));
  }

  errorEstructuraWison(lexema: string, linea: number, columna: number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en la estructura principal: Wison ¿ ... ? Wison'
    );
  }

  errorBloqueLex(lexema: string, linea: number, columna: number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en bloque lexico: Lex {: ... :}'
    );
  }

  errorDeclaracionTerminal(lexema: string, linea: number, columna:
number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en declaracion: Terminal $_Nombre <- expresion ;'
    );
  }

  errorExpresionRegular(lexema: string, linea: number, columna: number)
{
    this.errorSintactico(lexema, linea, columna,
      'Expresion regular invalida'
    );
  }

  errorBloqueSyntax(lexema: string, linea: number, columna: number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en bloque sintactico: Syntax {{: ... :}}'
    );
  }

  errorDeclaracionNoTerminal(lexema: string, linea: number, columna:
number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en declaracion: No_Terminal %_Nombre ;'
    );
  }

  errorSimboloInicial(lexema: string, linea: number, columna: number) {
```

```
    this.errorSintactico(lexema, linea, columna,
      'Error en declaracion: Initial_Sim %_Nombre ;'
    );
  }

  errorProduccion(lexema: string, linea: number, columna: number) {
    this.errorSintactico(lexema, linea, columna,
      'Error en produccion: %_NoTerminal <= simbolos... ;'
    );
  }

  /* ERRORES SEMANTICOS */

  errorSemantico(lexema: string, linea: number, columna: number,
descripcion: string) {
    this.errores.push(new Token(
      lexema,
      linea,
      columna,
      descripcion,
      'SEMANTICO'
    ));
  }

  terminalDuplicado(nombre: string, linea: number, columna: number) {
    this.errorSemantico(nombre, linea, columna,
      `Terminal '${nombre}' ya fue declarado anteriormente`
    );
  }

  noTerminalDuplicado(nombre: string, linea: number, columna: number) {
    this.errorSemantico(nombre, linea, columna,
      `No terminal '${nombre}' ya fue declarado anteriormente`
    );
  }

  terminalNoDeclarado(nombre: string, linea: number, columna: number) {
    this.errorSemantico(nombre, linea, columna,
      `Terminal '${nombre}' usado pero no declarado`
    );
  }
```

```
 noTerminalNoDeclarado(nombre: string, linea: number, columna: number)
{
   this.errorSemantico(nombre, linea, columna,
     `No terminal '${nombre}' usado pero no declarado`
   );
 }

 simboloInicialNoDeclarado(nombre: string, linea: number, columna:
number) {
   this.errorSemantico(nombre, linea, columna,
     `Simbolo inicial '${nombre}' no fue declarado como No_Terminal`
   );
 }

 sinSimboloInicial() {
   this.errorSemantico('', 0, 0,
     'No se declaro un simbolo inicial con Initial_Sim'
   );
 }

 regexToString(nodo: any): string {

   if (!nodo) return "";

   switch (nodo.tipo) {

     case 'Caracter':
       return `'${nodo.valor}'`;

     case 'Rango':
       return nodo.valor;

     case 'ReferenciaTerminal':
       return nodo.nombre;

     case 'Union':
       return this.regexToString(nodo.izq) +
         " | " +
         this.regexToString(nodo.der);

     case 'Concatenacion':
       return this.regexToString(nodo.izq) +
         this.regexToString(nodo.der);
```

```
    case 'Kleene':
      return this.regexToString(nodo.expr) + "*";

    case 'CerraduraPositiva':
      return this.regexToString(nodo.expr) + "+";

    case 'Opcional':
      return this.regexToString(nodo.expr) + "?";

    case 'Grupo':
      return "(" + this.regexToString(nodo.expr) + ")";

    default:
      return "";
  }

}

produccionToString(nodo: any): string {

  if (!nodo) return "";

  if (nodo.epsilon) return "ε";

  switch (nodo.tipo) {

    case 'Secuencia':
      return nodo.simbolos
        .map((s: any) => s.nombre)
        .join(" ");

    case 'Alternativa':
      return nodo.opciones
        .map((o: any) => this.produccionToString(o))
        .join(" | ");

    default:
      return "";
  }

}
```

```
 getToken(): Token[] { return this.tokens; }
 getError(): Token[] { return this.errores; }

 getHayErrores(): boolean { return this.errores.length > 0; }

 get resumen() {
   return {
     totalTokens: this.tokens.length,
     totalErrores: this.errores.length,
     erroresLexicos: this.errores.filter(e => e.type ===
'LEXICO').length,
     erroresSintact: this.errores.filter(e => e.type ===
'SINTACTICO').length,
     erroresSemanticos: this.errores.filter(e => e.type ===
'SEMANTICO').length,
   };
 }

}
```

Genera una gramática en Wison

Wison ¿

Lex {:
  declaraciones de terminales
:}

Syntax {{:
  declaraciones de no terminales
  símbolo inicial
  producciones
:}}

?Wison

Lo que realiza el parser es que :

- exista el bloque léxico
- exista el bloque sintáctico
- exista un símbolo inicial
- las producciones estén bien formadas

Forma de declarar la parte Lex.

```
# Declaración de terminales de la forma:
# Terminal $_NOMBRE <- EXPRESIÓN ;

Terminal $_Una_A     <- 'a' ;    # cualquier carácter alfanumérico por separado
Terminal $_Mas       <- '+' ;    # cualquier carácter especial por separado
Terminal $_Punto     <- '.' ;    # cualquier carácter especial por separado
Terminal $_P_Ab      <- '(' ;    # cualquier carácter especial por separado
Terminal $_P_Ce      <- ')' ;    # cualquier carácter especial por separado
Terminal $_FIN       <- 'FIN';   # cualquier palabra reservada
Terminal $_Letra     <- [aA-zZ]; # alfabeto completo en mayusculas y minusculas
Terminal $_NUMERO    <- [0-9] ;  # Digitos del 0 al 9
Terminal $_NUMEROS   <- [0-9]* ; # Estrella de Kleene para hacer 0 o n veces
Terminal $_NUMEROS_2 <- [0-9]+ ; # Cerradura positiva para hacer 1 on veces
Terminal $_NUMEROS_3 <- [0-9]? ; # reconoce la cláusula ? para hacer 0 o 1 vez
Terminal $_Decimal   <- ([0-9]*)($_Punto)($_NUMEROS_2); # terminal combinado
```

El parser reconoce el texto y lo vuelve un token y eso es lo que devuelve.

Forma de declarar la parte Syntax

```
Syntax {{:
   # Declaración de no terminales de la forma
   # No_Terminal %_Nombre ;

   No_Terminal %_Prod_A;
   No_Terminal %_Prod_B;
   No_Terminal %_Prod_C;
   No_Terminal %_S;


   # Simbolo inicial de la forma
   # Initial_Sim %_Nombre ;

   Initial_Sim %_S ;

   #Todo símbolo no terminal debe ser declarado antes de usarse en las producciones
   # Las producciones son de la siguiente forma
   # %_Initial_Sim  <= %_Prod_A ... %_No_terminal_N o $_Terminal_N ... ;
```

```
   %_S <= %_Prod_A $_FIN ;
   %_Prod_A <= $_P_Ab %_Prod_B $_P_Ce ;
   %_Prod_B <= %_Prod_B %Prod_C | %_Prod_C ;
   %_Prod_C <= $_Una_A $_Mas $_Una_A ;
:}}
```

El parser procesa mediante la producción:

```
declaracion_terminal

    : TERMINAL ID_TERMINAL ASIGNACION expresion_regular PUNTO_COMA
```

y cada terminal se almacena para la tabla de simbolos de esta manera

```
yy.manejador.agregarToken(

   nombre,

   linea,

   columna,

   "Expresión regular: ...",

   "TERMINAL"

);
```

La producción para las no terminales

```
declaracion_no_terminal

    : NO_TERMINAL ID_NO_TERMINAL PUNTO_COMA
```

Produccion para el simbolo inicial

```
simbolo_inicial

    : SIMBOLO_INICIAL ID_NO_TERMINAL PUNTO_COMA
```

El parser genera el AST con los distintos tipos:

- Secuencia
- Alternativa
- Terminal
- No terminal
- epsilon

y los guarda de la siguiente manera:

```
yy.manejador.agregarToken(

   textoProduccion,

   linea,
```

```
  columna,
  "Producción gramatical",
  "PRODUCCION"
);
```

Ejemplo de una gramatica Wison para tomar la idea:

Wison ¿

```
Lex {:
Terminal $_NUM <- [0-9]+ ;
Terminal $_MAS <- '+' ;
Terminal $_ID <- [aA-zZ]+ ;
:}
Syntax {{:
No_Terminal %_E ;
No_Terminal %_T ;

Initial_Sim %_E ;
%_E <= %_E $_MAS %_T | %_T ;
%_T <= $_NUM ;

:}}
```

?Wison

Esta gramática sencilla es para aceptar cadenas de suma como ejemplo:

5 + 5

5 + 5+ 5 +5